Chapter 7: Interlude – Bayesian Data Analysis

Authors: Michael Henry Tessler; Noah Goodman

Inference by conditioning a generative model is a basic building block of Bayesian statistics. In cognitive science this tool can be used in two ways:

- If the generative model is a hypothesis about a person's model of the world, then we have a Bayesian *cognitive model* – the main topic of this book.
- If the generative model is instead the scientist's model of how the data are generated, then we have *Bayesian Data Analysis*.

Bayesian Data Analysis can be an extremely useful tool to us as scientists, when we are trying to understand what our data "tell us" about psychological hypotheses.

This can become confusing, however: a particular modeling assumption can be something we hypothesize that people assume about the world, or can be something that we as scientists want to assume (but that we *don't* assume that people assume). A pithy way of saying this is that we can make assumptions about "Bayes in the head" (Bayesian cognitive models) or about "Bayes in the notebook" (Bayesian Data Analysis).

Bayesian Data Analysis (BDA) is a general-purpose approach to making sense of data. A BDA model is an **explicit hypotheses about the generative process behind the experimental data** – where did the observed data come from? For instance, we might hypothesize that data from two experimental conditions came from two *different* distributions.

After constructing these kinds of explicit hypotheses, Bayesian inference can be used to ***invert* the model**: to go from experimental **data** to updated **beliefs** about the hypotheses.

7.1 Parameters and Predictives

In a BDA model the random variables are usually called *parameters*. Parameters can be of theoretical interest, or not (the latter are called nuisance parameters). Parameters are in general unobservable (or, "latent"), so we must infer them from observed data. We can also go from updated beliefs about parameters to expectations about future data, in the form of so-called *posterior predictive* distributions.

For a given Bayesian model (together with data), there are **four conceptually distinct distributions** we often want to examine. For parameters, we have priors and posteriors:

- The ***Prior Distribution*** *over parameters* captures our initial state of knowledge (or beliefs) about the values that the latent parameters could have, before seeing the data.
- The ***Posterior Distribution*** *over parameters* captures what we know about the latent parameters having updated our beliefs with the evidence provided by data.

From either the prior or the posterior over parameters we can then run the model forward, to get **predictions** about data sets:

- The ***Prior Predictive*** *distribution* tells us what data to expect, given our model and our initial beliefs about the parameters. The prior predictive is a distribution over data, and gives the relative probability of different *observable* outcomes before we have seen any data.
- The ***Posterior Predictive*** *distribution* tells us what data to expect, given the same model we started with, but with beliefs that have been updated by the observed data. The posterior predictive is a distribution over data, and gives the relative probability of different observable outcomes, after some data has been seen.

Loosely speaking, *predictive* distributions are in "data space" and *parameter* distributions are in "latent parameter space".

7.1.1 Example: Election Surveys

Imagine you want to find out how likely Candidate A is to win an election. To do this, you will try to estimate the proportion of eligible voters in the United States who will vote for Candidate A in the election.

Trying to determine directly how many (voting age, likely to vote) people prefer Candidate A to Candidate B would require asking over 100 million people (it's estimated that about 130 million people voted in the US Presidential Elections in 2008 and 2012). Thus it is impractical to measure the whole distribution.

Instead, pollsters measure a sample (maybe ask 1000 people), and use that to draw conclusions about the "true population proportion" (an unobservable parameter). Here, we explore the result of an experiment with 20 trials and binary outcomes ("Will you vote for Candidate A or Candidate B?"):

```
// Observed data:
// Number of people who support candidate A
const k = 1
// Number of people surveyed
const n = 20
```

```
var model = function() {
  // true population proportion who support candidate A
  var p = uniform(0, 1)

  // Observed k people support "A"
  // Assuming each person's response is
  // independent of each other
  observe(
    Binomial({p : p, n: n}),
    k
  )

  // Predict what the next n will say
  var posteriorPredictive = binomial(p, n)

  // Recreate model structure, without observe
  var prior_p = uniform(0, 1)
  var priorPredictive = binomial(prior_p, n)

  return {
    prior: prior_p,
    priorPredictive: priorPredictive,
    posterior: p,
    posteriorPredictive: posteriorPredictive
  }
}
var posterior = Infer(
  {method: 'rejection'},
  model
)
viz.marginals(posterior)
```

Notice that some plots densities and others bar graphs. This is because the **data space** for this experiment is based on counts, while the **parameter space** is continuous.

What can we conclude intuitively from examining these plots?

- First, because prior differs from posterior (both parameters and predictives), the **evidence** has **changed our beliefs**.
- Second, the posterior predictive assigns quite high probability to the true data, suggesting that the model considers the data "reasonable".

- Finally, after observing the data, we conclude that the true proportion of people supporting Candidate A is quite low: around 0.09, or at least somewhere between 0.0 and 0.15.

Check your understanding by trying other data sets, varying both `k` and `n`.

7.2 Quantifying Claims About Parameters

How can we quantify a claim like "the true parameter is low"? One possibility is to compute the *mean* or *expected value* of the parameter, which is mathematically given by $\int xp(x)dx$ for a posterior distribution $p(x)$.

In WebPPL this can be computed via `expectation(d)` for a distribution `d`. Thus in the above election example we could do the following:

```
// Observed data
// Number of people who support candidate A
const k = 1
// Number of people surveyed
const n = 20

var model = function() {
  // True population proportion who support candidate A
  var p = uniform(0, 1)

  // Observed k people support "A"
  // Assuming each person's response is
  // independent of each other
  observe(
    Binomial({p : p, n: n}),
    k
  )

  // Predict what the next n will say
  var posteriorPredictive = binomial(p, n)

  // Recreate model structure, without observe
  var prior_p = uniform(0, 1)
  var priorPredictive = binomial(prior_p, n)
  return p
}

var posterior = Infer(
  {method: 'rejection'},
```

```
  model
)
print('')
print("Expected proportion voting for A: " + expectation(posterior) )
```

This tells us that the mean is about 0.09. This can be a very useful way to summarize a posterior, but it eliminates crucial information about how *confident* we are in this mean. A coherent way to summarize our confidence is by exploring the probability that the parameter lies within a given interval. Conversely, an interval that the parameter lies in with high probability (say 90%) is called a *credible interval* (CI). Let's explore credible intervals for the parameter in the above model:

```
// Observed data
// Number of people who support candidate A
const k = 1
// Number of people surveyed
const n = 20

var model = function() {
  // true population proportion who support candidate A
  var p = uniform(0, 1)

  // Observed k people support "A"
  // Assuming each person's response is
  // independent of each other
  observe(
    Binomial({p : p, n: n}),
    k
  )

  // Predict what the next n will say
  var posteriorPredictive = binomial(p, n)

  // Recreate model structure, without observe
  var prior_p = uniform(0, 1)
  var priorPredictive = binomial(prior_p, n)
  return p
}

var posterior = Infer(
  {method: 'rejection'},
  model
```

```
)

// Compute the probability that p lies
// in the interval [0.01,0.18]
expectation(
  posterior,
  function(p){
    return (0.01 < p) && (p < 0.18)
  }
)
```

Here we see that $[0.01, 0.18]$ is an (approximately) 90% credible interval – we can be about 90% sure that the true parameter lies within this interval. Notice that the 90% CI is not unique. There are different ways to choose a particular CI.

One particularly common, and useful, one is the Highest Density Interval (HDI), which is the smallest interval achieving a given confidence. (For unimodal distributions the HDI is unique and includes the mean.) Here is a quick way to approximate the HDI of a distribution:

```
var cred = function(d,low,up) {
  expectation(
    d,
    function(p) {
      return (low < p) && (p < up)
    }
  )
}

var findHDI = function(targetp, d, low, up, eps) {
  if (cred(d, low, up) < targetp) {
    return [low, up]
  }
  var y = cred(d, low + eps, up)
  var z = cred(d, low, up - eps)
  return y > z
    ? findHDI(targetp, d, low + eps, up, eps)
    : findHDI(targetp, d, low, up - eps, eps)
}

//simple test: a censored gaussian:
var d = Infer(
  function() {
```

```
    var x = gaussian(0, 1)
    condition(x > 0)
    return x
  }
)

//find the 95% HDI for this distribution:
findHDI(0.95, d, -10, 10, 0.1)
```

Credible intervals are related to, but shouldn't be mistaken for, the confidence intervals that are often used in frequentist statistics.[1]

7.2.1 Example: Logistic Regression

Now imagine you are a pollster who is interested in the effect of age on voting tendency. You run the same poll, but you are careful to recruit people in their 20s, 30s, etc. We can analyze this data by assuming there is an underlying linear relationship between age and voting tendency.

However, since our data are boolean (or, if we aggregate, the count of people in each age group who will vote for candidate A), we must use a function to link from real numbers to the range [0,1]. The logistic function is a common way to do so:

```
var data = [
  {age: 20, n:20, k:1},
  {age: 30, n:20, k:5},
  {age: 40, n:20, k:17},
  {age: 50, n:20, k:18}
]

var logistic = function(x) {
  return (1 / (1 + Math.exp(-x)))
}

var model = function() {
  // true effect of age
  var a = gaussian(0,1)
  //true intercept
  var b = gaussian(0,1)

  // observed data
```

[1] Moreover, these confidence intervals themselves should definitely not be confused with p-values!

```
  map(
    function(d){
      observe(
        Binomial({
          p: logistic(a*d.age + b),
          n: d.n
        }),
        d.k
      )
    },
    data
  )

  return {
    a: a,
    b: b
  }
}

var opts = {method: "MCMC", samples: 50000}
var posterior = Infer(opts, model)
viz.marginals(posterior)
```

Looking at the parameter posteriors we see that there seems to be an effect of age: the parameter `a` is greater than zero, so older people are more likely to vote for candidate A. But how well does the model really explain the data?

7.3 Posterior Prediction and Model Checking

The posterior predictive distribution describes what data you should **expect to see**, given the **model you've assumed** and the **data you've collected** so far.

If the model is able to describe the data you've collected, then the model shouldn't be surprised if you got the same data by running the experiment again. That is, the most likely data for your model after observing your data should be the data you observed. It is natural then to use the posterior predictive distribution to examine the descriptive adequacy of a model. If these predictions do not match the data *already seen* (i.e., the data used to arrive at the posterior distribution over parameters), the model is descriptively inadequate.

A common way to check whether the posterior predictive matches the data, imaginatively called a *posterior predictive check*, is to plot some statistics of your data vs the expectation of

these statistics according to the predictive. Let's do this for the number of votes in each age group:

```
var data = [
  {age: 20, n:20, k:1},
  {age: 30, n:20, k:5},
  {age: 40, n:20, k:17},
  {age: 50, n:20, k:18}
]
var ages = map(
  function(d) {
    return d.age
  },
  data
)

var logistic = function(x) {
  return (1 / (1 + Math.exp(-x)))
}

var model = function() {
  // true effect of age
  var a = gaussian(0,1)
  //true intercept
  var b = gaussian(0,1)

  // observed data
  map(
    function(d){
      observe(
        Binomial({
          p: logistic(a*d.age + b),
          n: d.n
        }),
        d.k
      )
    },
    data
  )

  // posterior prediction for each age in
  // data, for n=20:
  var predictive = map(
```

```
    function(age){
      binomial({
        p: logistic(a * age + b),
        n: 20
      })
    },
    ages
  )

  return predictive
}

var opts = {method: "MCMC", samples: 50000}
var posterior = Infer(opts, model)
var ppstats = mapIndexed(
  function(i,a) {
    expectation(
      posterior,
      function(p){
        p[i]
      }
    )
  },
  ages
)
var datastats = map(
  function(d){
    return d.k
  },
  data
)
viz.scatter(ppstats, datastats)
```

This scatter plot will lie on the $x = y$ line if the model completely accomodates the data.

Looking closely at this plot, we see that there are a few differences between the data and the predictives:

- First, the predictions are compressed compared to the data. This is likely because the prior encourages moderate probabilities.
- Second, we see hints of non-linearity in the data that the model is not able to account for.

This model is pretty good, but certainly not perfect, for explaining this data.

One final note: There is an important, and subtle, difference between a model's ability to *accomodate* some data and that model's ability to *predict* the data. This is roughly the difference between a *posterior* predictive check and a *prior* predictive check.

7.4 Model Selection

In the above examples, we've had a *single* data-analysis model and used the experimental data to learn about the parameters of the models and the descriptive adequacy of the models.

Often as scientists, we are in fortunate position of having *multiple*, distinct models in hand, and want to decide if one or another is a better description of the data. This problem, the problem of *model selection* given data, is one of the most important and difficult tasks of BDA.

Returning to the simple election polling example above, imagine we begin with a (rather unhelpful) data analysis model that assumes each candidate is equally likely to win, that is $p = 0.5$.

We quickly notice, by looking at the posterior predictives, that this model doesn't accommodate the data well at all. We thus introduce the above model where $p \sim \text{Uniform}(0, 1)$. How can we quantitatively decide which model is better?

One approach is to combine the models into a "meta model" that decides which approach to take:

```
// Observed data
// Number of people who support candidate A
const k = 5
// Number of people surveyed
const n = 20

var model = function() {
  // binary decision variable for
  // which hypothesis is better
  var x = flip(0.5)
    ? "simple"
    : "complex"
  var p = (x == "simple")
    ? 0.5
    : uniform(0, 1)

  observe(
    Binomial({
      p: p,
      n: n
```

```
    }),
    k
  )
  return {
    model: x
  }
}
var modelPosterior = Infer(
  {method: 'rejection'},
  model
)
viz(modelPosterior)
```

We see that, as expected, the more complex model is preferred: we can confidently say that given the data the more complex model is the one we should believe. Further we can quantify this via the posterior probability of the complex model.

This model is an example from the classical hypothesis testing framework. We consider a model that fixes one of its parameters to a pre-specified value of interest (here $\mathcal{H}_0 : p = 0.5$). This is sometimes referred to as a *null hypothesis*. The other model says that the parameter is free to vary. In the classical hypothesis testing framework, we would write: $\mathcal{H}_1 : p \neq 0.5$.

With Bayesian hypothesis testing, we must be explicit about what p is (not just what p is not), so we write $\mathcal{H}_1 : p \sim \text{Uniform}(0, 1)$.

One might have a conceptual worry: Isn't the second model just a more general case of the first model? That is, if the second model has a uniform distribution over `p`, then `p: 0.5` is included in the second model. Fortunately, the posterior on models automatically penalizes the more complex model when it's flexibility isn't needed. (To verify this, set `k=10` in the example.)

This idea is called the principle of parsimony or Occam's razor, and will be discussed at length later in this book. For now, it's sufficient to know that more complex models will be penalized for being more complex, intuitively because they will be diluting their predictions. At the same time, more complex models are more flexible and can capture a wider variety of data (they are able to bet on more horses, which increases the chance that they will win some money). Bayesian model comparison lets us weigh these costs and benefits.

7.4.1 The Bayes Factor

What we are plotting above are *posterior model probabilities*. These are a function of the marginal likelihoods of the data under each hypothesis and the prior model probabilities (here, defined to be equal: `flip(0.5)`).

Sometimes, scientists feel a bit strange about reporting values that are based on prior model probabilities (what if scientists have different priors as to the relative plausibility of the hypotheses?) and so often report the ratio of marginal likelihoods, a quantity known as a *Bayes Factor*.

Let's compute the Bayes Factor, by computing the likelihood of the data under each hypothesis:

```
const k = 5
const n = 20

var simpleModel = Binomial({
  p: 0.5,
  n: n
})

var complexModel = Infer(
  function() {
    var p = uniform(0, 1);
    return binomial(p, n)
  }
)

var simpleLikelihood = Math.exp(simpleModel.score(k))
var complexLikelihood = Math.exp(complexModel.score(k))

var bayesFactor = simpleLikelihood / complexLikelihood
// Clear buffer -JJ
print('')
print(bayesFactor)
```

How does the Bayes Factor in this case relate to posterior model probabilities above? A short derivation shows that when the model priors are equal, the Bayes Factor has a simple relation to the model posterior.

7.4.2 Savage-Dickey Method

Sometimes the Bayes factor can be obtained by computing marginal likelihoods directly. (As in the example, where we marginalize over the model parameter using `Infer`.) However, it is sometimes hard to get *good* estimates of the two marginal probabilities.

In the case where one model is a special case of the other, called *nested model comparison*, there is another option.

The Bayes factor can also be obtained by considering *only* the more complex hypothesis, by looking at the distribution over the parameter of interest (here, p) at the point of interest (here, $p = 0.5$). Dividing the probability density of the posterior by the density of the prior (of the parameter at the point of interest) gives you the Bayes Factor!

This (perhaps surprising!) result was described by Dickey and Lientz (1970), who attributed it to Leonard "Jimmie" Savage. The method is called the *Savage-Dickey density ratio* and is widely used in experimental science.

We would use it like so:

```
const k = 5
const n = 20

var complexModelPrior = Infer(
  function() {
    var p = uniform(0, 1)
    return p
  }
)

var complexModelPosterior = Infer(
  function() {
    var p = uniform(0, 1)
    observe(
      Binomial({p: p, n: n}),
      k
    )
    return p
  }
)

var savageDickeyDenomenator = expectation(
  complexModelPrior,
  function(x) {
    return (0.45 < x) && (x < 0.55)
  }
)
var savageDickeyNumerator = expectation(
  complexModelPosterior,
  function(x) {
    return (0.45 < x) && (x < 0.55)
  }
)
```

```
var savageDickeyRatio = (
  savageDickeyNumerator
  / savageDickeyDenomenator
)
print('')
print(savageDickeyRatio)
```

(Note that we have approximated the densities by looking at the expectation that p is within 0.05 of the target value $p = 0.5$.)

7.5 BDA of Cognitive Models

In this chapter we have described how we can use generative models of data to do data analysis.

In the rest of this book we are largely interested in how we can build *cognitive models* by hypothesizing that people have generative models of the world that they use to reason and learn. That is, we view people as intuitive Bayesian statisticians, doing in their heads what scientists do in their notebooks.

Of course when we, as scientists, try to test our cognitive models of people, we can do so using BDA! This leads to more complex models in which we have an "outer" Bayesian data analysis model and an "inner" Bayesian cognitive model. (This is exactly the "nested Infer" pattern introduced for Social cognition):

- What is in the inner (cognitive) model captures our scientific hypotheses about what people know and how they reason.
- What is in the outer (BDA) model represents aspects of our hypotheses about the data that we *are not* attributing to people: linking functions, unknown parameters of the cognitive model, and so on.

This distinction can be subtle!

7.6 Readings and Discussion

For further reading on Bayesian data analysis see:

- Lee and Wagenmakers (2013)
- Kruschke (2015)
- Gelman et al. (2013)

Dickey, James M., and B. P. Lientz. 1970. "The Weighted Likelihood Ratio, Sharp Hypotheses about Chances, the Order of a Markov Chain." *The Annals of Mathematical Statistics* 41 (1): 214–26.

Gelman, Andrew, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. 2013. *Bayesian Data Analysis*. CRC Press.

Kruschke, John. 2015. *Doing Bayesian Data Analysis*. Elsevier.

Lee, Michael D., and Eric-Jan Wagenmakers. 2013. *Bayesian Cognitive Modeling: A Practical Course*. Cambridge University Press.

<script> document.addEventListener("DOMContentLoaded", () => document.querySelectorAll('pre').forEach((elt) => editor.setup(elt, language:'webppl'))); </script>